SQL Injection SEED Lab

Matthew Blanchard

CYEN 3331-01

3/22/26

Executive Summary:

The objective of this laboratory was to identify, exploit, and subsequently mitigate SQL injection vulnerabilities within a localized web application environment. By leveraging a Docker-based infrastructure, various attack vectors were explored, including authentication bypass, unauthorized data retrieval, and the malicious modification of database records such as salaries and passwords. The findings demonstrated that direct concatenation of user input into SQL queries allows attackers to manipulate backend logic, leading to total account takeover and data exposure. The project concluded with the successful implementation of prepared statements as a primary defense mechanism, effectively neutralizing injection attempts by separating query logic from user-provided data.

Introduction:

SQL injection remains one of the most critical vulnerabilities in web applications, occurring when untrusted data is sent to an interpreter as part of a command or query. This report documents a series of practical exercises conducted within a controlled SEED Lab environment to understand the mechanics of these attacks and their real-world implications. The study begins with a foundational exploration of SQL syntax and database structures before progressing to active exploitation through both web and command-line interfaces. Key areas of focus include authentication bypass, unauthorized data manipulation, and the application of industry-standard countermeasures like prepared statements to secure vulnerable source code. By simulating these attacks, the lab provides a comprehensive look at how architectural choices in database communication directly impact the security posture of an entire system.

Lab Environment Setup

1. Open the /etc/hosts file using root privileges and add the entry 10.9.0.5 www.seed-server.com.
2. Navigate to the Labsetup directory within the terminal.
3. Execute the command docker-compose build to create the necessary container images.
4. Run docker-compose up to start the web server and database containers.
5. Use docker ps to find the ID of the MySQL container.
6. Open a shell inside the container by executing docker exec -it <id> /bin/bash.

Task 1: Get Familiar with SQL Statements

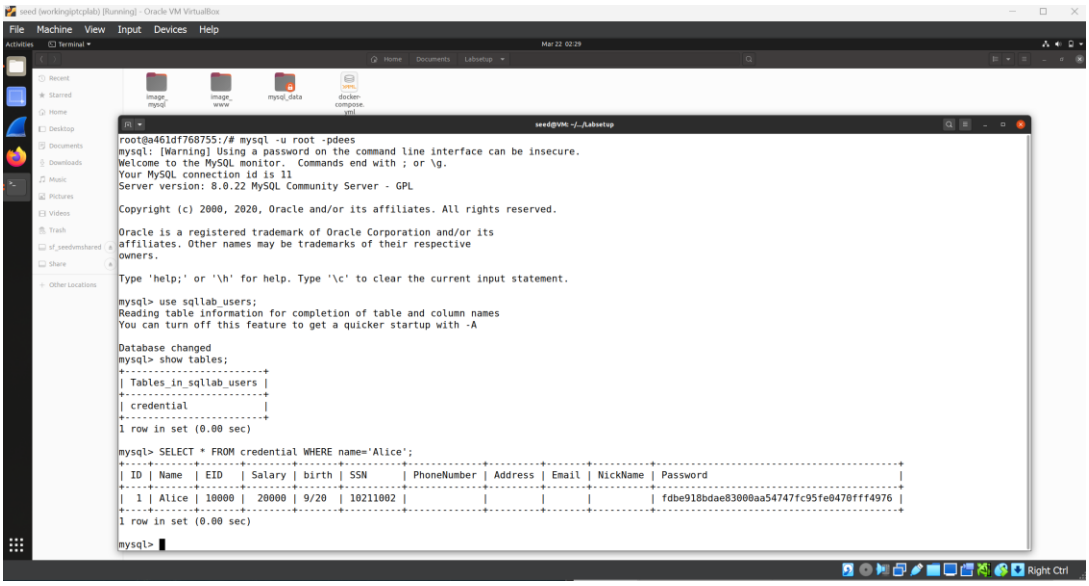
Objective:

The objective is to interact directly with the MySQL database via the command-line interface to understand the underlying data structure. Accessing the credential table allows for the verification of stored user profiles and sensitive fields such as salaries and passwords.

Instructions:

1. Access the MySQL command-line interface by typing mysql -u root -pdees.
2. Load the specific database for the lab using the command use sqllab_users;.
3. List the available tables in the database by executing show tables;.
4. Retrieve Alice's profile data by running SELECT * FROM credential WHERE name='Alice';.
5. Observe the table output to identify columns like salary, eid, and ssn.

Observations:



Running the command to select the table from the correct database and then selecting Alice yeilds a table with all of the information associated with Alice.

Task 2: SQL Injection Attack on SELECT Statement

Objective:

This task focuses on exploiting vulnerabilities in SELECT queries to bypass authentication and retrieve unauthorized data. By manipulating the input logic, the security constraints of the login mechanism are circumvented.

Task 2.1: Attack from Webpage

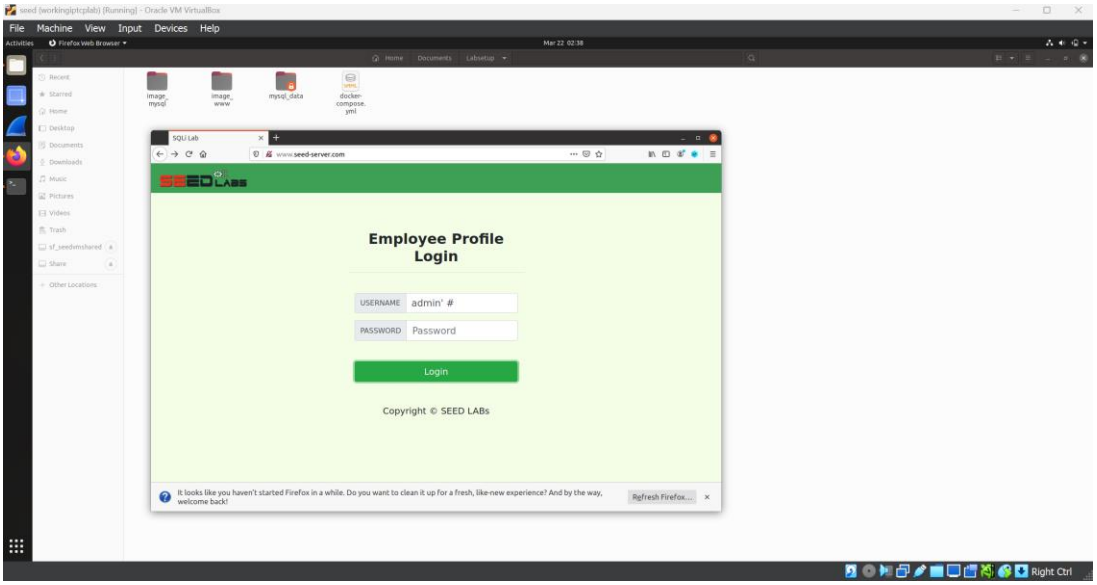
Objective:

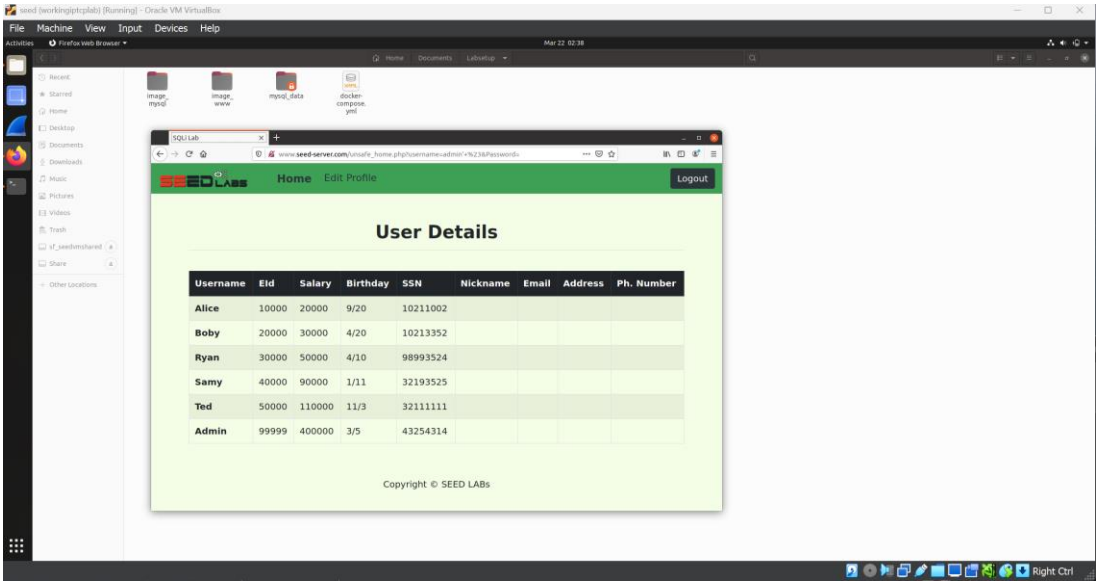
The goal is to use the web-based login form to perform an authentication bypass using a comment-based injection. Successful execution demonstrates how an attacker can log in as an administrator without possessing a valid password.

Instruction:

1. Navigate to http://www.seed-server.com in a web browser.
2. Enter admin' # into the Username field and leave the Password field empty.
3. Click the login button to execute the modified query.
4. Observe that the application grants access to the administrative account without a valid password.
5. Verify that the browser displays the private information for all employees in the database.

Observations:





After analyzing the file unsafe_home.php, the condition for ‘admin’ can be located. Using this information, the command ‘admin’ #’ can be put into the user input field. The result is all users with their associated information are displayed.

Task 2.2: Attack from Command Line

Objective:

The objective is to replicate the authentication bypass using curl to understand how web requests are structured at the protocol level. This subtask highlights the importance of URL-encoding special characters to ensure the malicious payload is processed correctly by the server.

Instructions:

1. Identify the URL and GET parameters required for the login request.
2. Construct a curl command using the target URL http://www.seed-server.com/unsafe_home.php.
3. Manually URL-encode the payload by replacing the single quote with %27, the space with %20, and the hash with %23.
4. Wrap the entire URL in single quotes to prevent the terminal shell from misinterpreting the special characters.
5. Execute the command and inspect the returned HTML to confirm the data was successfully retrieved.

Observations:

seed (workingiptcplab) [Running] - Oracle VM VirtualBox

File Machine View Input Devices Help

seed@VM: ~/.../Labsetup

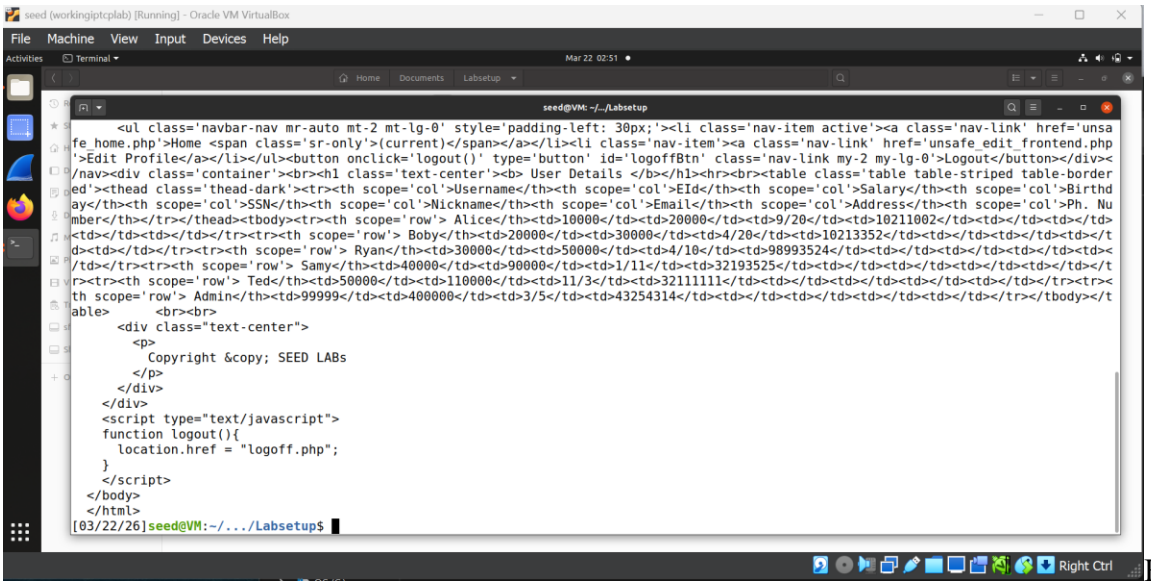
```
[03/22/26]seed@VM:~/.../Labsetup$ curl 'http://www.seed-server.com/unsafe_home.php?username=admin%27+%23&Password='
<!--
SEED Lab: SQL Injection Education Web plateform
Author: Kailiang Ying
Email: kying@syr.edu
-->

<!--
SEED Lab: SQL Injection Education Web plateform
Enhancement Version 1
Date: 12th April 2018
Developer: Kuber Kohli

Update: Implemented the new bootsrap design. Implemented a new Navbar at the top with two menu options for Home and edit profile, with a
 button to
logout. The profile details fetched will be displayed using the table class of bootstrap with a dark table head theme.

NOTE: please note that the navbar items should appear only for users and the page with error login message should not have any of these
items at
all. Therefore the navbar tag starts before the php tag but it end within the php script adding items as required.
-->

<!DOCTYPE html>
<html lang="en">
<head>
  <!-- Required meta tags -->
  <meta charset="utf-8">
```

seed (workingiptcplab) [Running] - Oracle VM VirtualBox

File Machine View Input Devices Help

seed@VM: ~/.../Labsetup

```
logout. The profile details fetched will be displayed using the table class of bootstrap with a dark table head theme.

NOTE: please note that the navbar items should appear only for users and the page with error login message should not have any of these
items at
all. Therefore the navbar tag starts before the php tag but it end within the php script adding items as required.
-->

<!DOCTYPE html>
<html lang="en">
<head>
  <!-- Required meta tags -->
  <meta charset="utf-8">
  <meta name="viewport" content="width=device-width, initial-scale=1, shrink-to-fit=no">

  <!-- Bootstrap CSS -->
  <link rel="stylesheet" href="css/bootstrap.min.css">
  <link href="css/style_home.css" type="text/css" rel="stylesheet">

  <!-- Browser Tab title -->
  <title>SQLi Lab</title>
</head>
<body>
  <nav class="navbar fixed-top navbar-expand-lg navbar-light" style="background-color: #3EA055;">
    <div class="collapse navbar-collapse" id="navbarTogglerDemo01">
      <a class="navbar-brand" href="unsafe_home.php" ><img src="seed_logo.png" style="height: 40px; width: 200px;" alt="SEEDLabs"></a>

      <ul class='navbar-nav mr-auto mt-2 mt-lg-0' style='padding-left: 30px;'><li class='nav-item active'><a class='nav-link' href='unsa
```



Here

Using curl in the terminal, the url from the previous task that contains the ‘admin’ #’ input can be copied to be pasted after the curl command. Doing so gives the source code for the page showing the same information as the previous task, albeit in a much more cluttered, difficult to read manner. This method has its benefits, however, as it can be ran without needing to visit the webpage itself.

Task 2.3: Append a New SQL Statement

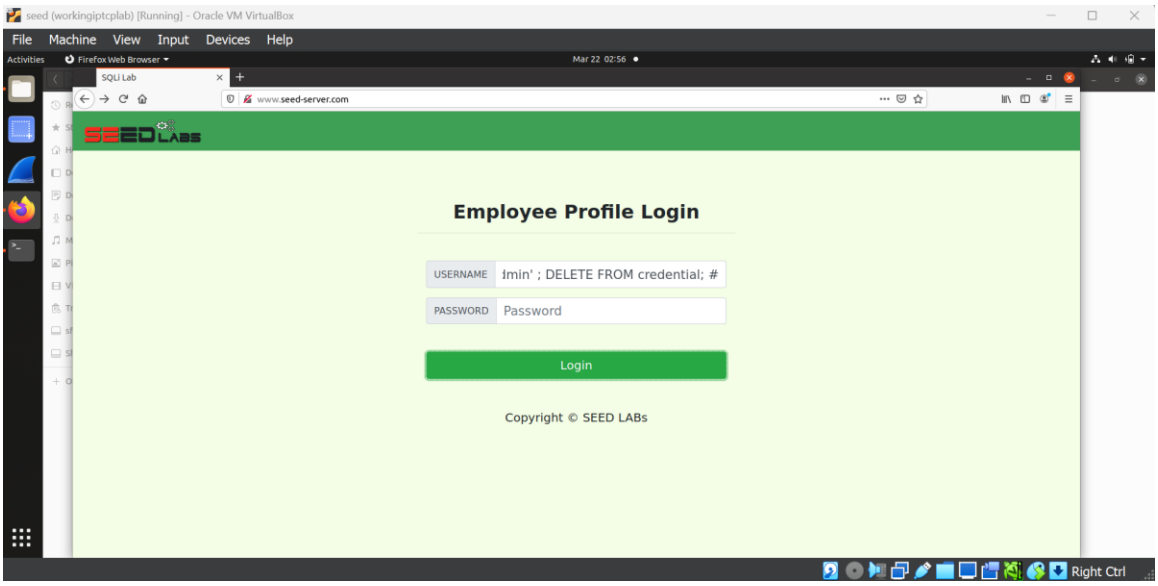
Objective:

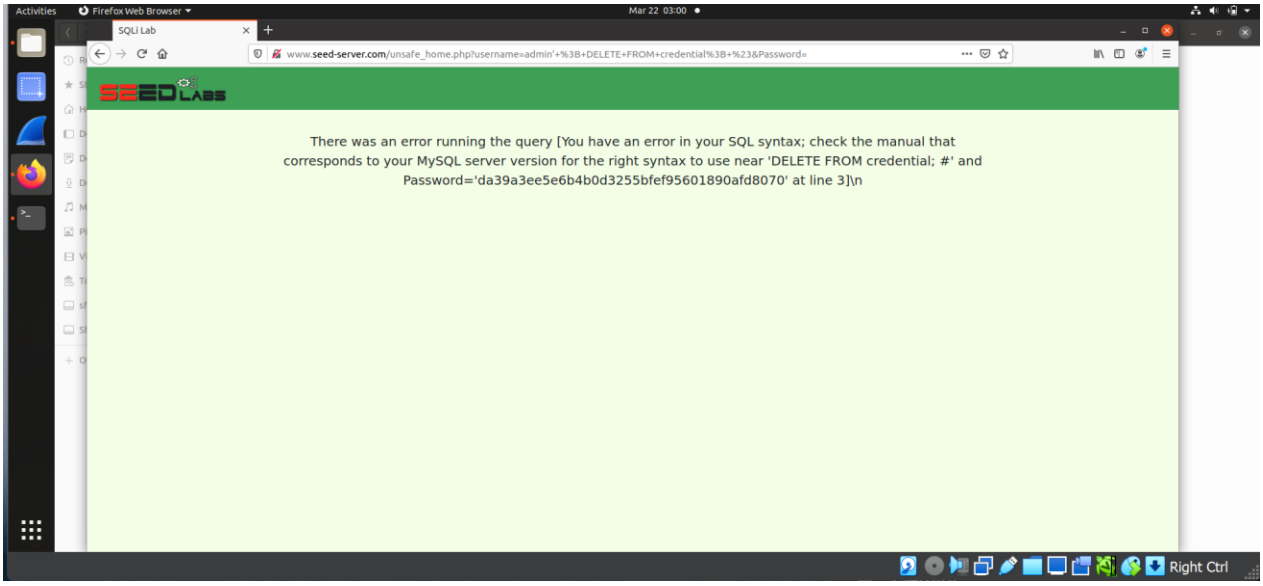
This subtask explores the limitations of certain PHP database functions regarding "stacked queries" or multiple statements. The objective is to observe how the mysqli::query() function acts as an accidental defense by refusing to execute more than one command per call.

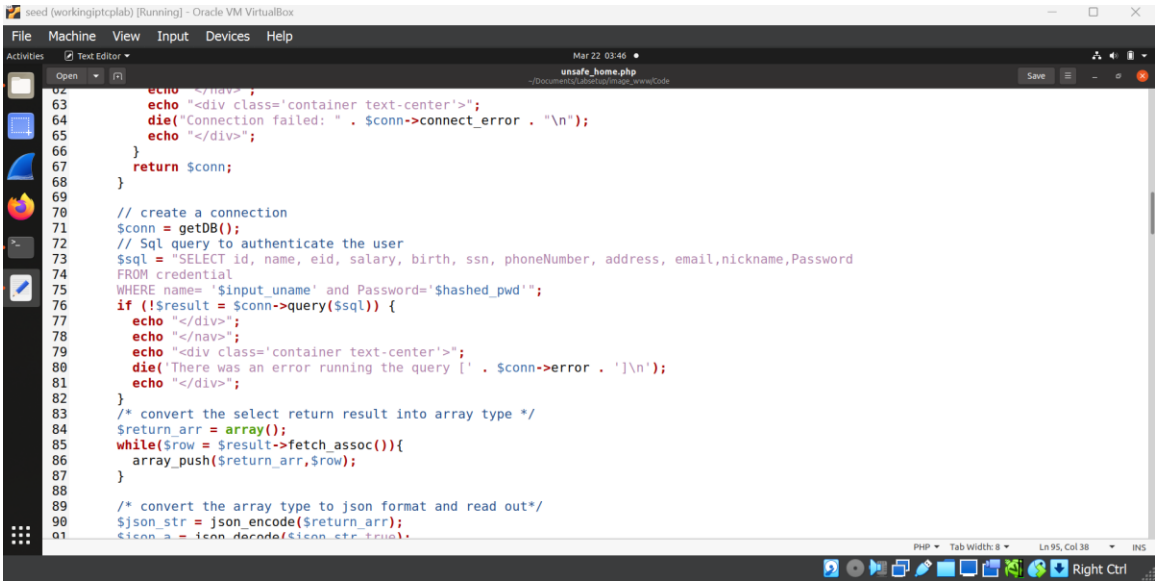
Instructions:

1. Attempt to log in by injecting a second SQL statement separated by a semicolon, such as admin'; update credential set salary=40000; #.
2. Observe the resulting error message or failure of the second command to execute.
3. Review the unsafe_home.php source code to identify the use of the mysqli::query() function.
4. Note that this specific PHP function is restricted to executing only one SQL statement at a time.

Observations:


seed (workingiptcplab) [Running] - Oracle VM VirtualBox
File Machine View Input Devices Help
Activities
Firefox Web Browser
Mar 22 02:56
SQLi Lab
www.seed-server.com
Employee Profile Login
USERNAME
admin' ; DELETE FROM credential; #
PASSWORD
Password
Login
Copyright © SEED LABs
Right Ctrl


Activities
Firefox Web Browser
Mar 22 03:00
SQLi Lab
www.seed-server.com/unsafe_home.php?username=admin'+%3B+DELETE+FROM+credential%3B+%23&Password=
There was an error running the query [You have an error in your SQL syntax; check the manual that corresponds to your MySQL server version for the right syntax to use near 'DELETE FROM credential; #' and Password='da39a3ee5e6b4b0d3255bfef95601890afd8070' at line 3]\n
Right Ctrl



Trying to run more than one SQL command results in a syntax error because as soon as the backend detects more than one command in the input, it throws an error as it cannot handle more than one command as the input. This is a built-in function as a result of the SQL API. In this case, this would be considered a helpful function as it prevents obvious attacks from multi-command inputs.

Task 3: SQL Injection Attack on UPDATE Statement

Objective:

The objective is to move beyond data retrieval and demonstrate how SQL injection can be used to modify existing database records. This involves identifying vulnerable UPDATE queries that do not properly sanitize user input.

Task 3.1: Modify Own Salary

Objective:

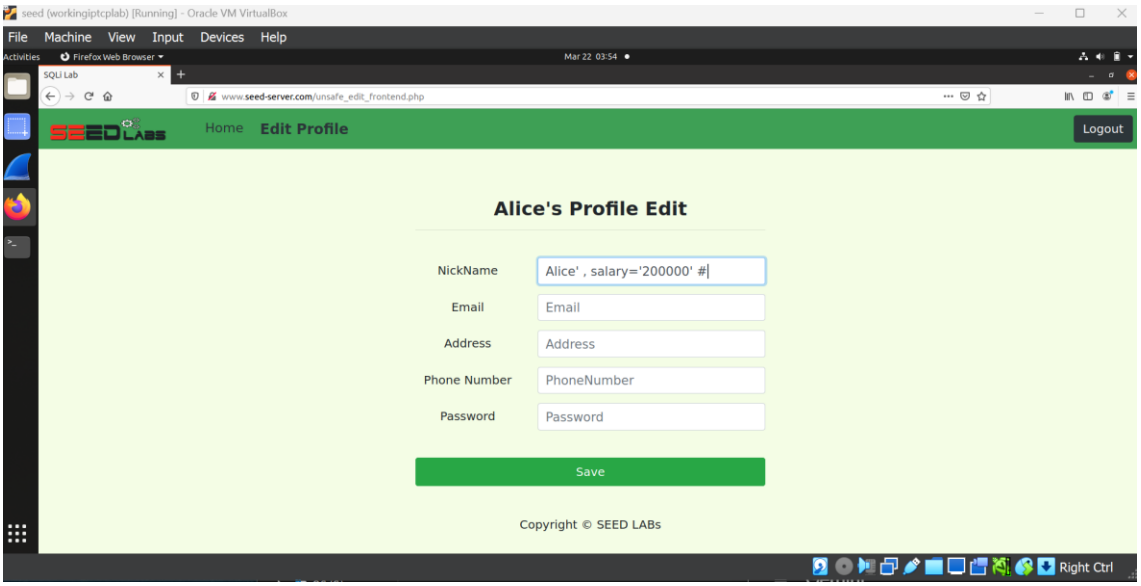
The goal is to manipulate the profile update form to change a specific column, such as salary, that is not normally accessible through the user interface. By injecting into a standard text field, the backend SQL statement is altered to include additional assignment logic.

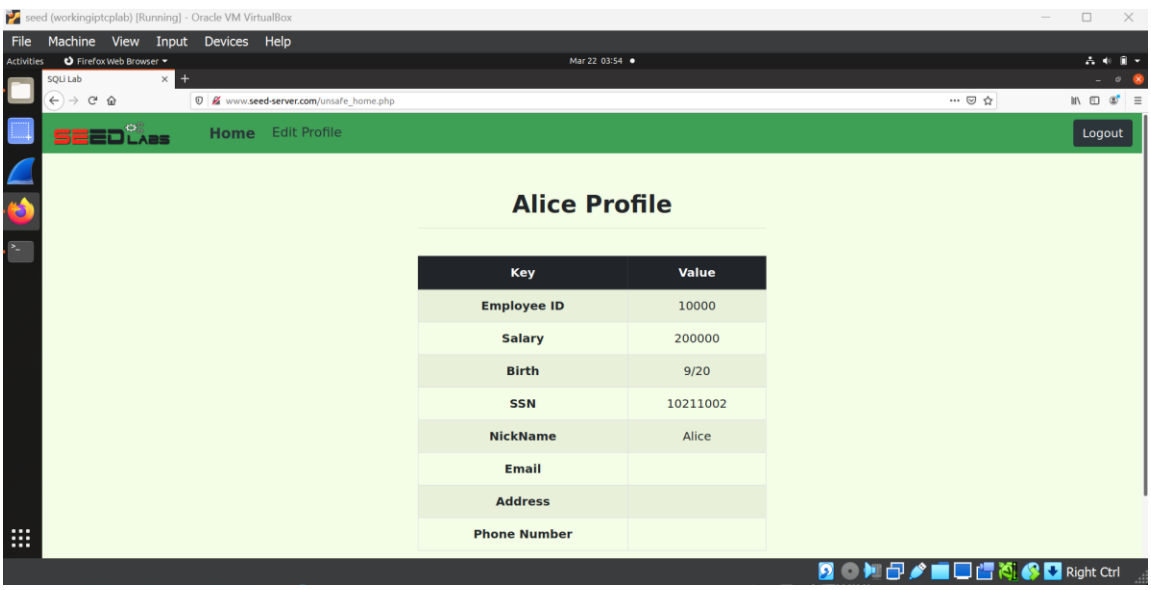
Instructions:

1. Log into the web application using Alice's legitimate credentials.
2. Navigate to the "Edit Profile" page to access the update form.
3. Enter the payload Alice', salary='50000' # into the Nickname input field.

4. Submit the form to trigger the UPDATE query on the backend.
5. Verify the increase in salary by viewing the profile or checking the database directly.

Observations:





Analyzing the unsafe_backend.php file and the file from previous tasks, the syntax can be determined to be largely the same to change the salary amount with the only difference being the key words. In this case, the command, ‘Alice’ , salary=’200000’ #’ is used in any of the edit profile input fields that are not hashed (all but the password field). Doing this changed the salary sucessfully as is reflected int eh updated profile page.

Task 3.2: Modify Other People’s Salary

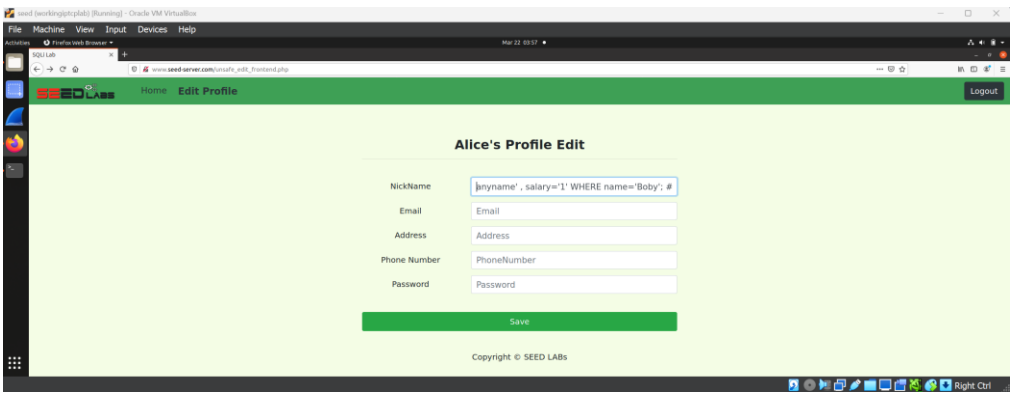
Objective:

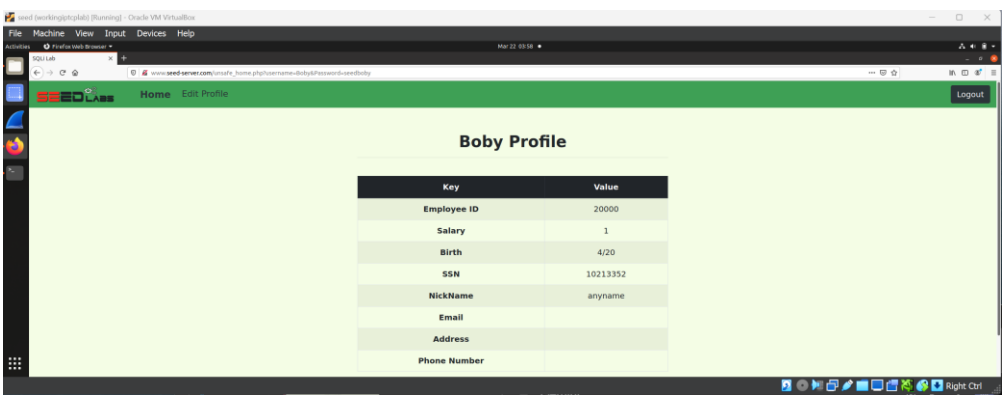
This subtask aims to escalate the attack by using a WHERE clause to modify the records of other users in the database. Successful execution proves that a vulnerability in one user's profile edit page can compromise the data integrity of the entire organization.

Instructions:

1. Use the Edit Profile page as an authenticated user to inject into the UPDATE statement.
2. Craft a payload that closes the first assignment and introduces a WHERE clause to target another user.
3. Input ', salary='1' WHERE name='Boby'; # into a text field like Nickname or Address.
4. Save the changes and verify that Boby's salary has been updated to 1 dollar in the database.

Observations:





This time, the approach is similar to the previous task; however, instead of specifying the user, it is necessary to query the database for any user that matches the desired name, Boby. The salary part is still the same as before. Doing this, the salary of Boby is successfully reduced.

Task 3.3: Modify Other People's Password

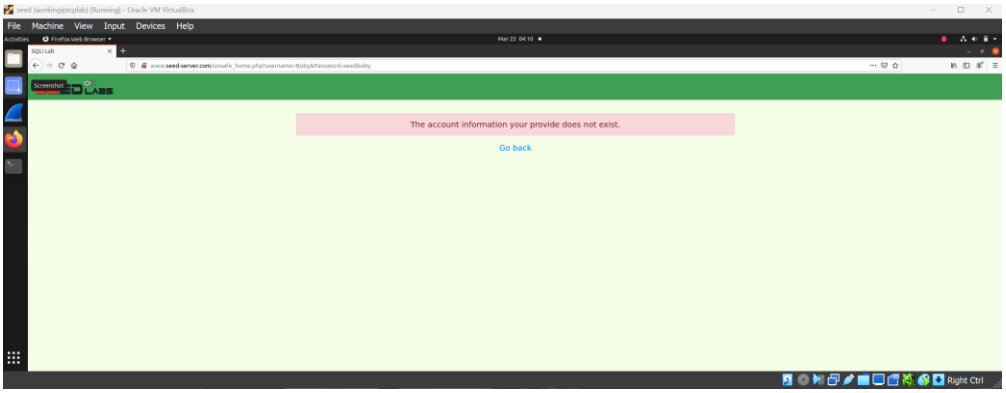
Objective:

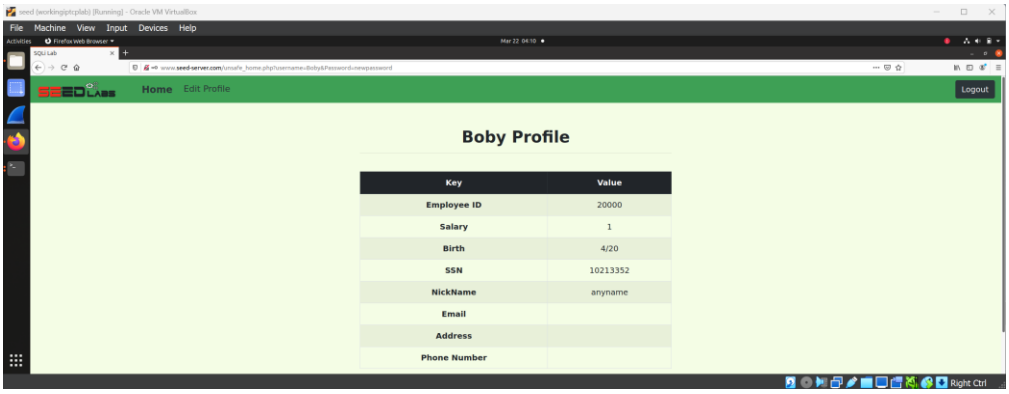
The objective is to achieve full account takeover by overwriting another user's password hash with a known value. Since the password field itself is often hashed by the application, the attack must be routed through a non-hashed field like a nickname.

Instructions:

1. Access the Edit Profile page while logged in as any user.
2. Locate a text field that is not hashed by the application, such as Nickname or Address.
3. Enter the payload ', Password=SHA1('newpassword') WHERE name='Boby'; # into the chosen field.
4. Submit the update to overwrite Boby's password hash with the hash of "newpassword".
5. Log out and attempt to log back in as Boby using the password newpassword to confirm the account takeover.

Observations:





Analyzing the file unsafe_edit_backend.php, the method for saving and hashing passwords can be found and exploited. Similar to previous tasks, the command used can be inputted into any text field on the 'Edit Profile" page that is not hashed. Again, the database is queried for any entries named Boby, pulling all of the data related to Boby. In this case, the command changes the passoword associated with the user 'Boby" using the SHA1 hash function

found in the file. Sucessfully executing this command renders the old password ineffective while the new password is effective.

Task 4: Countermeasure (Prepared Statement)

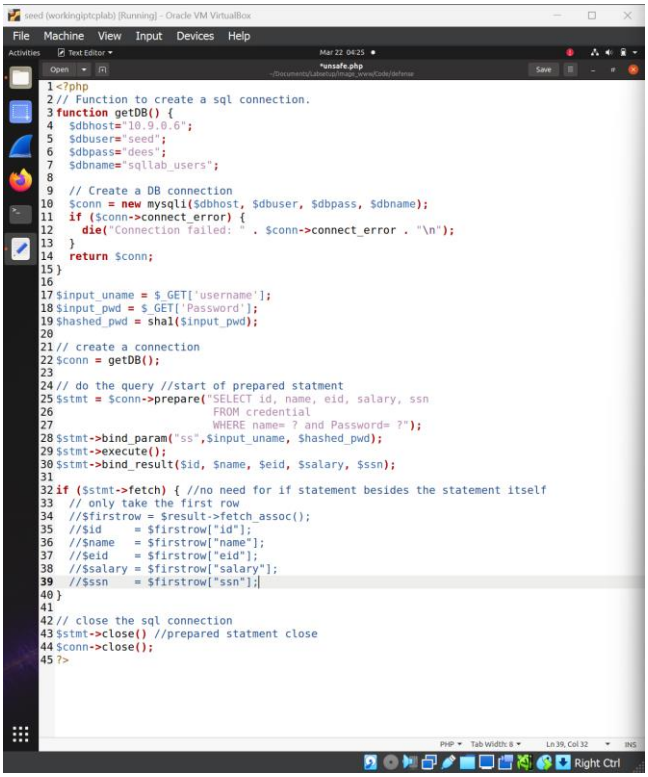
Objective:

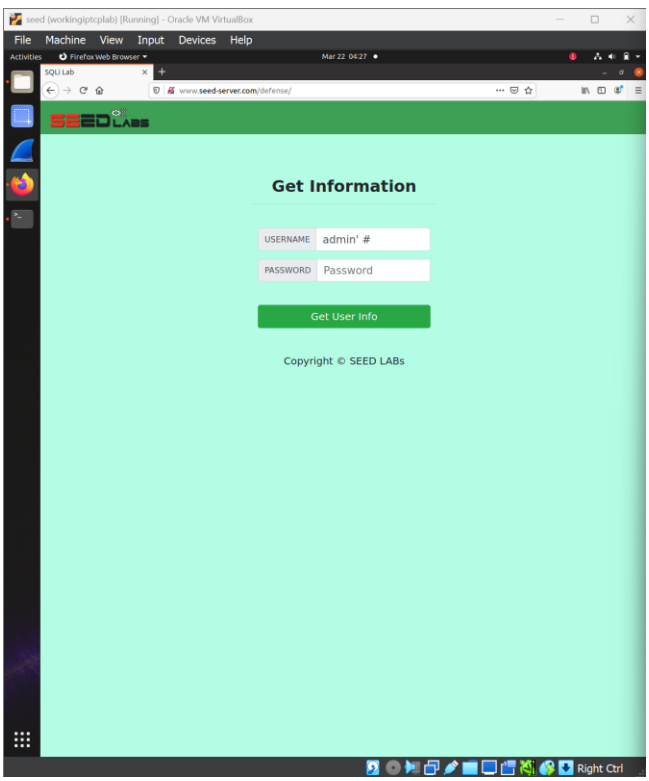
The primary goal is to implement a robust defense mechanism using prepared statements to neutralize SQL injection vulnerabilities. By separating the SQL logic from the user data, the database is prevented from interpreting malicious input as executable code.

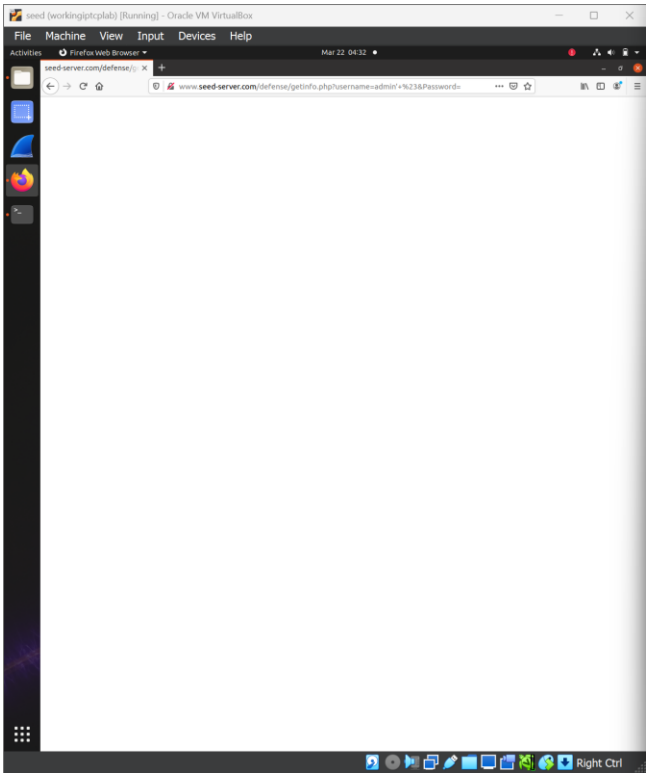
Instructions:

1. Access the defense version of the application at http://www.seed-server.com/defense/.
2. Open the file /var/www/SQL_Injection/defense/unsafe.php for editing.
3. Modify the SQL string to use ? placeholders for the name and Password values.
4. Replace the direct query() call with $conn->prepare() to initialize the statement template.
5. Use $stmt->bind_param("ss", $input_uname, $hashed_pwd) to bind user inputs as literal strings.
6. Execute the statement using $stmt->execute() and bind the output columns to variables.
7. Rebuild the environment by running docker-compose build and docker-compose up.
8. Attempt the admin' # injection on the defense page to confirm that the attack is no longer successful.

Observations:


seed (workingiptcplab) [Running] - Oracle VM VirtualBox
File Machine View Input Devices Help
Activities Text Editor Mar 22 04:25
Open *unsafe.php Save
1 <?php
2 // Function to create a sql connection.
3 function getDB() {
4 $dbhost="10.9.0.6";
5 $dbuser="seed";
6 $dbpass="dees";
7 $dbname="sqllab_users";
8
9 // Create a DB connection
10 $conn = new mysqli($dbhost, $dbuser, $dbpass, $dbname);
11 if ($conn->connect_error) {
12 die("Connection failed: " . $conn->connect_error . "\n");
13 }
14 return $conn;
15 }
16
17 $input_uname = $_GET['username'];
18 $input_pwd = $_GET['Password'];
19 $hashed_pwd = sha1($input_pwd);
20
21 // create a connection
22 $conn = getDB();
23
24 // do the query //start of prepared statment
25 $stmt = $conn->prepare("SELECT id, name, eid, salary, ssn
26 FROM credential
27 WHERE name= ? and Password= ?");
28 $stmt->bind_param("ss",$input_uname, $hashed_pwd);
29 $stmt->execute();
30 $stmt->bind_result($id, $name, $eid, $salary, $ssn);
31
32 if ($stmt->fetch) { //no need for if statement besides the statement itself
33 // only take the first row
34 //$firstrow = $result->fetch_assoc();
35 //$id = $firstrow["id"];
36 //$name = $firstrow["name"];
37 //$eid = $firstrow["eid"];
38 //$salary = $firstrow["salary"];
39 //$ssn = $firstrow["ssn"];
40 }
41
42 // close the sql connection
43 $stmt->close() //prepared statment close
44 $conn->close();
45 ?>
PHP Tab Width: 8 Ln 39, Col 32 INS
Right Ctrl


seed (workingiptcplab) [Running] - Oracle VM VirtualBox
File Machine View Input Devices Help
Activities Firefox Web Browser Mar 22 04:27
SQLi Lab
www.seed-server.com/defense/
SEED LABS
Get Information
USERNAME admin' #
PASSWORD Password
Get User Info
Copyright © SEED LABs
Right Ctrl



Using the provided code as reference, the file unsafe.php is edited to use the prepared statement mechanism. Sucessfully implementing this new code renders the old injection of ‘admin’ #’ in the user field ineffective and instead results in a blank page as the database attempted to query the admin, but found nothing associated with it. This proves the effectiveness of prepared statements in defending against SQL injections.

Conclusion:

The successful execution of the tasks in this lab highlights the severe risks associated with insecure coding practices, specifically the reliance on unsanitized user input in database queries. Through the exploitation of SELECT and UPDATE statements, it was proven that even minor oversight in input handling can lead to catastrophic security breaches, including the exposure of sensitive employee data and the loss of administrative control. However, the final phase of the lab demonstrated that these vulnerabilities are entirely preventable through the use of modern security protocols. By transitioning from dynamic string concatenation to prepared statements, the application's attack surface was effectively removed, ensuring that the database treats all user input as literal data rather than executable code.